

```

import React, { useState, useRef, useEffect } from 'react';

const VERSION = "1.0";
const MANIFEST_NAME = "manifest.json";

// =====
// SHA-256 streaming implementation (純 JS，唔靠 WebCrypto)
// =====
class SHA256 {
  constructor() {
    this.h = new Uint32Array([
      0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
      0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
    ]);
    this.k = new Uint32Array([
      0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a
      0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a
      0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9d
      0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca635
      0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92
      0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e358
      0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4
      0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f
    ]);
    this.buf = new Uint8Array(64);
    this.bufLen = 0;
    this.totalLen = 0n;
  }
  _block(p, off) {
    const w = new Uint32Array(64);
    for (let i = 0; i < 16; i++) {
      w[i] = (p[off+4*i]<<24) | (p[off+4*i+1]<<16) | (p[off+4*i+2]<<8) | p[off+4*
    ]
    for (let i = 16; i < 64; i++) {
      const s0 = ((w[i-15]>>>7)|(w[i-15]<<25)) ^ ((w[i-15]>>>18)|(w[i-15]<<14)) ^
      const s1 = ((w[i-2]>>>17)|(w[i-2]<<15)) ^ ((w[i-2]>>>19)|(w[i-2]<<13)) ^ (w
      w[i] = (w[i-16] + s0 + w[i-7] + s1) | 0;
    }
    let a=this.h[0],b=this.h[1],c=this.h[2],d=this.h[3],e=this.h[4],f=this.h[5],g
    for (let i = 0; i < 64; i++) {
      const S1 = ((e>>>6)|(e<<26)) ^ ((e>>>11)|(e<<21)) ^ ((e>>>25)|(e<<7));
      const ch = (e & f) ^ ((~e) & g);
      const t1 = (h + S1 + ch + this.k[i] + w[i]) | 0;
      const S0 = ((a>>>2)|(a<<30)) ^ ((a>>>13)|(a<<19)) ^ ((a>>>22)|(a<<10));

```

```

    const mj = (a & b) ^ (a & c) ^ (b & c);
    const t2 = (s0 + mj) | 0;
    h=g; g=f; f=e; e=(d+t1)|0; d=c; c=b; b=a; a=(t1+t2)|0;
}
this.h[0]=(this.h[0]+a)|0; this.h[1]=(this.h[1]+b)|0;
this.h[2]=(this.h[2]+c)|0; this.h[3]=(this.h[3]+d)|0;
this.h[4]=(this.h[4]+e)|0; this.h[5]=(this.h[5]+f)|0;
this.h[6]=(this.h[6]+g)|0; this.h[7]=(this.h[7]+h)|0;
}
update(data) {
    if (!(data instanceof Uint8Array)) data = new Uint8Array(data);
    this.totalLen += BigInt(data.length);
    let off = 0;
    if (this.bufLen > 0) {
        const need = 64 - this.bufLen;
        if (data.length < need) {
            this.buf.set(data, this.bufLen); this.bufLen += data.length; return;
        }
        this.buf.set(data.subarray(0, need), this.bufLen);
        this._block(this.buf, 0);
        this.bufLen = 0;
        off = need;
    }
    while (off + 64 <= data.length) {
        this._block(data, off);
        off += 64;
    }
    const rem = data.length - off;
    if (rem > 0) {
        this.buf.set(data.subarray(off), 0);
        this.bufLen = rem;
    }
}
digestHex() {
    const bitLen = this.totalLen * 8n;
    const padLen = (this.bufLen < 56) ? (56 - this.bufLen) : (120 - this.bufLen);
    const tail = new Uint8Array(padLen + 8);
    tail[0] = 0x80;
    const view = new DataView(tail.buffer);
    view.setBigUint64(padLen, bitLen, false);
    this.update(tail);
    let out = "";
    for (let i = 0; i < 8; i++) {
        out += this.h[i].toString(16).padStart(8, "0");
    }
    return out;
}

```

```

}

// =====
// ZIP writer (STORE mode, ZIP64 support)
// =====
class ZipWriter {
    constructor() {
        this.entries = [];
        this.parts = [];
        this.offset = 0n;
        this.crcTable = ZipWriter._crcTable();
    }
    static _crcTable() {
        const t = new Uint32Array(256);
        for (let n = 0; n < 256; n++) {
            let c = n;
            for (let k = 0; k < 8; k++) c = (c & 1) ? (0xedb88320 ^ (c >>> 1)) : (c >>> 1);
            t[n] = c;
        }
        return t;
    }
    _crc32(data) {
        let c = 0xffffffff;
        for (let i = 0; i < data.length; i++) {
            c = this.crcTable[(c ^ data[i]) & 0xff] ^ (c >>> 8);
        }
        return (c ^ 0xffffffff) >>> 0;
    }
    _push(buf) { this.parts.push(buf); this.offset += BigInt(buf.length); }
    _enc(s) { return new TextEncoder().encode(s); }

    addFile(name, data) {
        const nameBytes = this._enc(name);
        const crc = this._crc32(data);
        const size = data.length;
        const useZip64 = size >= 0xffffffff;
        const entryOffset = this.offset;
        const lfh = new Uint8Array(30 + nameBytes.length + (useZip64 ? 20 : 0));
        const dv = new DataView(lfh.buffer);
        dv.setUint32(0, 0x04034b50, true);
        dv.setUint16(4, useZip64 ? 45 : 20, true);
        dv.setUint16(6, 0x0800, true);
        dv.setUint16(8, 0, true);
        dv.setUint16(10, 0, true);
        dv.setUint16(12, 0x21, true);
        dv.setUint32(14, crc, true);
        dv.setUint32(18, useZip64 ? 0xffffffff : size, true);
    }
}

```

```

dv.setUint32(22, useZip64 ? 0xffffffff : size, true);
dv.setUint16(26, nameBytes.length, true);
dv.setUint16(28, useZip64 ? 20 : 0, true);
lfh.set(nameBytes, 30);
if (useZip64) {
    const ev = new DataView(lfh.buffer, 30 + nameBytes.length, 20);
    ev.setUint16(0, 0x0001, true);
    ev.setUint16(2, 16, true);
    ev.setBigUint64(4, BigInt(size), true);
    ev.setBigUint64(12, BigInt(size), true);
}
this._push(lfh);
this._push(data);
this.entries.push({ name: nameBytes, size, crc, offset: entryOffset, useZip64
}

finalize() {
    const cdStart = this.offset;
    for (const e of this.entries) {
        const useZip64 = e.useZip64 || e.offset >= 0xffffffffn;
        const extraLen = useZip64 ? (4 + (e.size >= 0xffffffff ? 16 : 0) + (e.offse
        const cd = new Uint8Array(46 + e.name.length + extraLen);
        const dv = new DataView(cd.buffer);
        dv.setUint32(0, 0x02014b50, true);
        dv.setUint16(4, useZip64 ? 45 : 20, true);
        dv.setUint16(6, useZip64 ? 45 : 20, true);
        dv.setUint16(8, 0x0800, true);
        dv.setUint16(10, 0, true);
        dv.setUint16(12, 0, true);
        dv.setUint16(14, 0x21, true);
        dv.setUint32(16, e.crc, true);
        dv.setUint32(20, e.size >= 0xffffffff ? 0xffffffff : e.size, true);
        dv.setUint32(24, e.size >= 0xffffffff ? 0xffffffff : e.size, true);
        dv.setUint16(28, e.name.length, true);
        dv.setUint16(30, extraLen, true);
        dv.setUint16(32, 0, true);
        dv.setUint16(34, 0, true);
        dv.setUint16(36, 0, true);
        dv.setUint32(38, 0, true);
        dv.setUint32(42, e.offset >= 0xffffffffn ? 0xffffffff : Number(e.offset), t
        cd.set(e.name, 46);
        if (useZip64) {
            let p = 46 + e.name.length;
            const ev = new DataView(cd.buffer);
            ev.setUint16(p, 0x0001, true); ev.setUint16(p+2, extraLen - 4, true); p +
            if (e.size >= 0xffffffff) {
                ev.setBigUint64(p, BigInt(e.size), true); p += 8;

```

```

        ev.setBigUint64(p, BigInt(e.size), true); p += 8;
    }
    if (e.offset >= 0xffffffff) {
        ev.setBigUint64(p, e.offset, true);
    }
}
this._push(cd);
}
const cdSize = this.offset - cdStart;
const cdOffset = cdStart;
const needZ64 = (cdOffset >= 0xffffffff) || (cdSize >= 0xffffffff) || (this
if (needZ64) {
    const z64eocdr = new Uint8Array(56);
    const dv = new DataView(z64eocdr.buffer);
    dv.setUint32(0, 0x06064b50, true);
    dv.setBigUint64(4, 44n, true);
    dv.setUint16(12, 45, true);
    dv.setUint16(14, 45, true);
    dv.setUint32(16, 0, true);
    dv.setUint32(20, 0, true);
    dv.setBigUint64(24, BigInt(this.entries.length), true);
    dv.setBigUint64(32, BigInt(this.entries.length), true);
    dv.setBigUint64(40, cdSize, true);
    dv.setBigUint64(48, cdOffset, true);
    const z64loc = new Uint8Array(20);
    const dv2 = new DataView(z64loc.buffer);
    dv2.setUint32(0, 0x07064b50, true);
    dv2.setUint32(4, 0, true);
    dv2.setBigUint64(8, this.offset, true);
    dv2.setUint32(16, 1, true);
    this._push(z64eocdr);
    this._push(z64loc);
}
const eocd = new Uint8Array(22);
const dvE = new DataView(eocd.buffer);
dvE.setUint32(0, 0x06054b50, true);
dvE.setUint16(4, 0, true);
dvE.setUint16(6, 0, true);
dvE.setUint16(8, Math.min(this.entries.length, 0xffff), true);
dvE.setUint16(10, Math.min(this.entries.length, 0xffff), true);
dvE.setUint32(12, cdSize >= 0xffffffff ? 0xffffffff : Number(cdSize), true);
dvE.setUint32(16, cdOffset >= 0xffffffff ? 0xffffffff : Number(cdOffset), tr
dvE.setUint16(20, 0, true);
this._push(eocd);
return new Blob(this.parts, { type: "application/zip" });
}
}

```

```

// =====
// Helper functions
// =====
function fmtSize(n) {
  if (n === 0) return "0 B";
  const u = ["B", "KB", "MB", "GB", "TB"]; let i = 0; let v = n;
  while (v >= 1024 && i < u.length - 1) { v /= 1024; i++; }
  return v.toFixed(i === 0 ? 0 : 2) + " " + u[i];
}

function sanitizeFolder(name) {
  return name.replace(/[<>:"/\|?*\\x00]/g, "_").replace(/^[\\s.]+|[\\s.]+$ /g, "") |
}

// =====
// Splitter
// =====
async function splitFile(file, opts, onProgress) {
  const chunkBytes = Math.max(1, Math.round(opts.chunkMb * 1024 * 1024));
  const size = file.size;
  const result = {
    name: file.name,
    size,
    kind: "split",
    parts: [],
    manifest: null,
    packedBlob: null,
    error: null,
  };

  try {
    if (size === 0 || size <= chunkBytes) {
      result.kind = opts.copySmall ? "copy" : "skip";
      if (opts.copySmall) {
        const buf = size === 0 ? new Uint8Array(0) : new Uint8Array(await file.ar
        let sha = "";
        if (opts.verifyHash) {
          const h = new SHA256(); h.update(buf); sha = h.digestHex();
        }
        const partName = `${file.name}.part001`;
        const manifest = {
          manifest_version: 1,
          original_filename: file.name,
          original_extension: (file.name.includes(".")) ? "." + file.name.split("."
          original_size: size,
          chunk_size: chunkBytes,
          num_parts: size === 0 ? 0 : 1,

```

```

    sha256: sha,
    created_utc: new Date().toISOString().replace(/\\.d+Z$/, "Z"),
    tool: "filesplitter-artifact",
    note: "copied (under threshold)",
    parts: size === 0 ? [] : [{ name: partName, size: buf.length, sha256: s
  };
  const zw = new ZipWriter();
  if (size > 0) zw.addFile(partName, buf);
  zw.addFile(MANIFEST_NAME, new TextEncoder().encode(JSON.stringify(manifes
  result.parts = size === 0 ? [] : [{
    name: partName, size: buf.length, sha256: sha,
    blob: new Blob([buf], { type: "application/octet-stream" }),
  }];
  result.manifest = manifest;
  result.packedBlob = zw.finalize();
  result.zipName = sanitizeFolder(file.name) + ".zip";
}
onProgress(size, size);
return result;
}

const numParts = Math.ceil(size / chunkBytes);
const wholeHash = opts.verifyHash ? new SHA256() : null;
const parts = [];
let offset = 0;
let partIdx = 1;

while (offset < size) {
  const end = Math.min(offset + chunkBytes, size);
  const slice = file.slice(offset, end);
  const buf = new Uint8Array(await slice.arrayBuffer());
  const partName = `${file.name}.part${String(partIdx).padStart(3, "0")}`;
  let partSha = "";
  if (opts.verifyHash) {
    const h = new SHA256();
    h.update(buf);
    partSha = h.digestHex();
    wholeHash.update(buf);
  }
  parts.push({ name: partName, size: buf.length, sha256: partSha, data: buf });
  offset = end; partIdx++;
  onProgress(offset, size);
  await new Promise(r => setTimeout(r, 0));
}

const originalSha = opts.verifyHash ? wholeHash.digestHex() : "";
const manifest = {

```

```

    manifest_version: 1,
    original_filename: file.name,
    original_extension: (file.name.includes(".") ? "." + file.name.split(".").p
    original_size: size,
    chunk_size: chunkBytes,
    num_parts: parts.length,
    sha256: originalSha,
    created_utc: new Date().toISOString().replace(/\.d+Z$/, "Z"),
    tool: "filesplitter-artifact",
    parts: parts.map(p => ({ name: p.name, size: p.size, sha256: p.sha256 })),
  };
  const zw = new ZipWriter();
  for (const p of parts) zw.addFile(p.name, p.data);
  zw.addFile(MANIFEST_NAME, new TextEncoder().encode(JSON.stringify(manifest, n
  result.parts = parts.map(p => ({
    name: p.name, size: p.size, sha256: p.sha256,
    blob: new Blob([p.data], { type: "application/octet-stream" })),
  }));
  result.manifest = manifest;
  result.packedBlob = zw.finalize();
  result.zipName = sanitizeFolder(file.name) + ".zip";
  return result;
} catch (e) {
  result.kind = "error";
  result.error = e?.message || String(e);
  return result;
}
}

// =====
// React UI
// =====
export default function FileSplitter() {
  const [files, setFiles] = useState([]);
  const [chunkMb, setChunkMb] = useState(29.9);
  const [autoStart, setAutoStart] = useState(true);
  const [copySmall, setCopySmall] = useState(true);
  const [verifyHash, setVerifyHash] = useState(true);
  const [results, setResults] = useState([]);
  const [running, setRunning] = useState(false);
  const [currentProgress, setCurrentProgress] = useState({ idx: -1, percent: 0 })
  const [log, setLog] = useState(["檔案切割器 v" + VERSION + " 已載入。完全係你部機本地
  const fileInputRef = useRef(null);
  const autoStartRef = useRef(autoStart);
  useEffect(() => { autoStartRef.current = autoStart; }, [autoStart]);

  const addLog = (msg) => setLog(l => [...l, msg].slice(-100));

```



```

const handleFiles = (newFiles) => {
  const arr = Array.from(newFiles);
  if (arr.length === 0) return;
  setFiles(prev => {
    const seen = new Set(prev.map(f => f.name + "|" + f.size));
    const adds = arr.filter(f => !seen.has(f.name + "|" + f.size));
    const merged = [...prev, ...adds];
    addLog(`加入 ${adds.length} 個檔案 (共 ${merged.length} 個喺 queue) `);
    if (autoStartRef.current && adds.length > 0 && !running) {
      setTimeout(() => startSplitWith(merged), 50);
    }
    return merged;
  });
};

const startSplitWith = async (queueFiles) => {
  if (queueFiles.length === 0 || running) return;
  if (chunkMb >= 30) {
    if (!confirm(`你揀咗 ${chunkMb} MB ≥ 30 MB，多數 AI 平台會拒收。仲係要繼續？`)) r
  }
  setRunning(true);
  setResults([]);
  addLog(`開始切割：${queueFiles.length} 個檔案，每片 ${chunkMb} MB`);

  const opts = { chunkMb, copySmall, verifyHash };
  const out = [];
  for (let i = 0; i < queueFiles.length; i++) {
    const f = queueFiles[i];
    setCurrentProgress({ idx: i, percent: 0 });
    addLog(`[${i+1}/${queueFiles.length}] ${f.name}`);
    const r = await splitFile(f, opts, (cur, total) => {
      setCurrentProgress({ idx: i, percent: total > 0 ? cur * 100 / total : 100
    });
    out.push(r);
    setResults([...out]);
    if (r.kind === "error") addLog(`    ✕ 失敗: ${r.error}`);
    else if (r.kind === "skip") addLog(`    略過 (細過 chunk) `);
    else if (r.kind === "copy") addLog(`    細檔案，已複製到 zip`);
    else addLog(`    ✓ 切咗 ${r.parts.length} 片`);
  }
  setCurrentProgress({ idx: -1, percent: 0 });
  setRunning(false);
  setFiles([]);
  const nSplit = out.filter(r => r.kind === "split").length;
  const nCopy = out.filter(r => r.kind === "copy").length;
  const nFail = out.filter(r => r.kind === "error").length;

```

```

    addLog(`完成: ${nSplit} 切, ${nCopy} 複製, ${nFail} 失敗`);
  };

const handleDrop = (e) => {
  e.preventDefault();
  setDragOver(false);
  if (e.dataTransfer?.files) handleFiles(e.dataTransfer.files);
};

const [dragOver, setDragOver] = useState(false);

const downloadBlob = (blob, name) => {
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url; a.download = name;
  document.body.appendChild(a); a.click();
  document.body.removeChild(a);
  setTimeout(() => URL.revokeObjectURL(url), 30000);
};

return (
  <div className="min-h-screen bg-gradient-to-b from-slate-900 via-slate-900 to-
    <div className="max-w-2xl mx-auto pb-20">
      <header className="mb-5 pt-2">
        <h1 className="text-3xl font-bold bg-gradient-to-r from-orange-300 to-o
          檔案切割器
        </h1>
        <p className="text-sm text-slate-400 mt-1">
          切大檔案做 < 30 MB 嘅碎片, 方便上傳去 AI 工具。100% 喺你部機本地處理。
        </p>
      </header>

      { /* 揀檔案 */ }

      <section className="bg-slate-800/60 border border-slate-700 rounded-xl p-
        <h2 className="text-xs font-semibold uppercase tracking-wider text-slat
        <label
          onDragEnter={ (e) => { e.preventDefault(); setDragOver(true); }}
          onDragOver={ (e) => { e.preventDefault(); setDragOver(true); }}
          onDragLeave={ () => setDragOver(false) }
          onDrop={ handleDrop }
          className={ `block border-2 border-dashed rounded-xl p-6 text-center c
            dragOver ? "border-orange-400 bg-orange-400/5" : "border-slate-700
          }` }
        >
          <div className="text-4xl mb-2">📁</div>
          <div className="text-base font-semibold">揀呢度揀檔案</div>
          <div className="text-xs text-slate-400 mt-1">iPad / Mac 仲可以拖檔案入嚟
          <input

```

```

        ref={fileInputRef}
        type="file"
        multiple
        className="hidden"
        onChange={(e) => { handleFiles(e.target.files); e.target.value = ""
    />
</label>
{files.length > 0 && (
    <div className="mt-3 text-xs text-slate-400">
        Queue 共 {files.length} 個檔案
        <button
            onClick={() => { setFiles([]); addLog("已清空 queue"); }}
            className="ml-3 px-3 py-1 bg-slate-800 border border-slate-700 ro
        >清空</button>
    </div>
    )}
</section>

{/* 設定 */}
<section className="bg-slate-800/60 border border-slate-700 rounded-xl p-
    <h2 className="text-xs font-semibold uppercase tracking-wider text-slat
    <div className="flex items-center gap-3 mb-3">
        <label className="text-sm text-slate-400 w-24">每片大細</label>
        <input
            type="number" step="0.1" min="0.1" max="2048"
            value={chunkMb}
            onChange={(e) => setChunkMb(parseFloat(e.target.value) || 29.9)}
            className="bg-slate-900 border border-slate-700 rounded px-3 py-2 w
        />
        <span className="text-xs text-slate-400">MB (預設 29.9) </span>
    </div>
    <Toggle checked={autoStart} onChange={setAutoStart} label="揀完自動開始"
    <Toggle checked={copySmall} onChange={setCopySmall} label="細檔案都包埋落!
    <Toggle checked={verifyHash} onChange={setVerifyHash} label="計 SHA256 !
</section>

{/* 開始 */}
<section className="bg-slate-800/60 border border-slate-700 rounded-xl p-
    <button
        onClick={() => startSplitWith(files)}
        disabled={files.length === 0 || running}
        className="w-full py-3 rounded-lg font-bold text-base disabled:opacit
            bg-gradient-to-br from-orange-300 to-orange-500 text-orang
            shadow-lg shadow-orange-500/20 active:translate-y-px"
    >
        {running ? "切割中..." : "▶ 開始切割"}
    </button>

```

```

</section>

{/* 進度 */}
{running && currentProgress.idx >= 0 && (
  <section className="bg-slate-800/60 border border-slate-700 rounded-xl
    <h2 className="text-xs font-semibold uppercase tracking-wider text-slate-400
    <div className="text-sm mb-2">處理緊第 {currentProgress.idx + 1} 個檔案
    <div className="h-2 bg-slate-900 rounded-full overflow-hidden">
      <div
        className="h-full bg-gradient-to-r from-orange-300 to-orange-500
        style={{ width: `${currentProgress.percent}%` }}
      />
    </div>
  </section>
)}

{/* 結果 + 下載 */}
{results.length > 0 && (
  <section className="bg-slate-800/60 border border-slate-700 rounded-xl
    <h2 className="text-xs font-semibold uppercase tracking-wider text-slate-400
    <div className="flex flex-col gap-2">
      {results.map((r, i) => (
        <div key={i} className="bg-slate-900/60 border border-slate-700 rounded-xl
          <div className="text-sm font-semibold truncate">{r.name}</div>
          <div className="text-xs text-slate-400 mt-1 mb-2">
            {fmtSize(r.size)} · {" "}
            {r.kind === "error" ? <span className="text-red-400">失敗: {r.
              r.kind === "skip" ? "略過" :
              r.kind === "copy" ? "原檔複製" :
              `已切 ${r.parts.length} 片`}
          </div>
          {r.packedBlob && (
            <div className="flex flex-wrap gap-2">
              <button
                onClick={() => downloadBlob(r.packedBlob, r.zipName)}
                className="text-xs bg-orange-500/15 border border-orange-
              >
                📄 下載 {r.zipName} ({fmtSize(r.packedBlob.size)})
              </button>
              {r.parts.map(p => (
                <button
                  key={p.name}
                  onClick={() => downloadBlob(p.blob, p.name)}
                  className="text-xs bg-slate-800 border border-slate-700
                >
                  {p.name}
                </button>
              )}
            </div>
          )}
        </div>
      )}
    </div>
  )}
)

```

```

    )))
    {r.manifest && (
      <button
        onClick={() => downloadBlob(
          new Blob([JSON.stringify(r.manifest, null, 2)], {type
            "manifest.json"
          })
        className="text-xs bg-slate-800 border border-slate-700
      >
        manifest.json
      </button>
    )}
  </div>
)}
</div>
)))
</div>
</section>
)}

{/* 記録 */}
<section className="bg-slate-800/60 border border-slate-700 rounded-xl p-
  <h2 className="text-xs font-semibold uppercase tracking-wider text-slat
  <div className="bg-black/60 border border-slate-800 rounded p-3 text-xs
    {log.join("\n")}
  </div>
</section>
</div>
</div>
);
}

```

```

function Toggle({ checked, onChange, label, sub }) {
  return (
    <label className="flex items-start gap-3 py-2 cursor-pointer select-none">
      <button
        type="button"
        onClick={() => onChange(!checked)}
        className={`shrink-0 w-11 h-6 rounded-full border transition relative ${
          checked ? "bg-orange-400 border-orange-400" : "bg-slate-900 border-slat
        }`}
      >
        <span
          className={`absolute top-0.5 left-0.5 w-5 h-5 bg-white rounded-full tra
            checked ? "translate-x-5" : ""
          }`}
        />

```

```
    </button>
    <div className="flex-1">
      <div className="text-sm">{label}</div>
      {sub && <div className="text-xs text-slate-500">{sub}</div>}
    </div>
  </label>
);
}
```